MODUL 17 · BAB VI · ALGORITMA GRAF

Breadth-First Search (BFS)

Menelusuri graf lapis demi lapis untuk menemukan lintasan dengan jumlah busur paling sedikit dari satu simpul awal.

TUJUAN PEMBELAJARAN

Setelah mempelajari modul ini, kamu diharapkan mampu:

- Menjelaskan cara kerja BFS dan peran antrean (FIFO) di dalamnya.
- Menjelaskan arti atribut $u.color$, $u.d$, dan $u.\pi$ serta arti warna WHITE, GRAY, BLACK.
- Menelusuri prosedur BFS pada graf, termasuk isi antrean di setiap langkah.
- Menyusun lintasan terpendek (dalam jumlah busur) dari atribut predecessor.
- Menjelaskan running time $O(V + E)$.

PRASYARAT

- Modul 16: graf dan adjacency-list.

BAB / MATERI ASAL

Materi 11.1 ALGORITMA GRAF, BFS

POSISI DALAM URUTAN BELAJAR

Algoritma penelusuran graf pertama. Dibandingkan dengan DFS di Modul 18; konsep jarak dan predecessor dipakai lagi pada Modul 22–24.

DAFTAR ISI

1. Menyebar seperti Riak Air 1
2. Prosedur BFS 1
3. Penelusuran BFS dari Simpul s 3
4. Analisis Running Time dan Sifat BFS 7
5. Kesimpulan 8
- Kuis Pemahaman 9
- Kunci Jawaban dan Pembahasan 10
- Referensi 10

PENDAHULUAN

1. Menyebar seperti Riak Air

Jika sebutir batu dijatuhkan ke kolam, riaknya menyebar melingkar: titik-titik terdekat tersentuh lebih dulu, lalu yang lebih jauh, dan seterusnya. **Breadth-First Search** (BFS, penelusuran melebar) bekerja dengan cara serupa pada graf. Dimulai dari satu simpul awal s, BFS mengunjungi semua tetangga s (jarak 1), lalu semua tetangga mereka yang belum dikunjungi (jarak 2), dan seterusnya, lapis demi lapis.

Karena simpul dikunjungi berdasarkan urutan jarak, BFS langsung memberi jawaban atas pertanyaan penting: **berapa jumlah busur paling sedikit untuk mencapai setiap simpul dari s, dan lewat lintasan mana?** Pertanyaan ini muncul pada banyak persoalan nyata, misalnya mencari jumlah perpindahan minimum di jaringan transportasi atau "derajat pertemanan" di media sosial.

Modul ini membahas prosedur BFS dari bahan kuliah baris demi baris, menelusurinya pada graf delapan simpul, dan menganalisis running time-nya.

PEMBAHASAN INTI

2. Prosedur BFS

Masukan prosedur BFS adalah graf G (direpresentasikan sebagai adjacency-list) dan simpul awal (*source vertex*) s.

2.1 Atribut setiap simpul

Atribut	Arti
$u.color$	Warna simpul u: **WHITE** = belum ditemukan; **GRAY** = sudah ditemukan dan sedang berada di antrean (tetangganya belum selesai diperiksa); **BLACK** = sudah selesai (semua tetangganya sudah diperiksa).
$u.d$	Jarak dari s ke u, berupa jumlah busur pada lintasan yang ditemukan. Awalnya ∞.
$u.\pi$	*Predecessor* (parent) u: simpul yang menemukan u. Untuk s dan simpul yang belum ditemukan, nilainya NIL.

Tabel 1. Atribut simpul pada BFS.

2.2 Antrean dan dua fungsinya

- `ENQUEUE(Q, v)` : memasukkan simpul v (yang baru diberi warna GRAY) ke bagian belakang antrean Q.
- `DEQUEUE(Q)` : mengeluarkan satu simpul dari bagian depan Q, mengikuti aturan **First In First Out** (FIFO): yang pertama masuk, pertama keluar.

Aturan FIFO inilah yang membuat BFS melebar: simpul berjarak k selalu masuk antrean sebelum simpul berjarak $k+1$, sehingga juga dikeluarkan lebih dulu.

BFS(G, s)

```
1  for each vertex u ∈ G.V - {s}
2      u.color = WHITE
3      u.d = ∞
4      u.π = NIL
5  s.color = GRAY
6  s.d = 0
7  s.π = NIL
8  Q = ∅
9  ENQUEUE(Q, s)
10 while Q ≠ ∅
11     u = DEQUEUE(Q)
12     for each v ∈ G.Adj[u]
13         if v.color == WHITE
14             v.color = GRAY
15             v.d = u.d + 1
16             v.π = u
17             ENQUEUE(Q, v)
18     u.color = BLACK
```

Langkah	Penjelasan
1–4	Setiap simpul selain s diberi warna WHITE, jarak ∞, dan predecessor NIL.
5–7	Simpul awal s diberi warna GRAY, jarak 0, predecessor NIL.
8–9	Antrean Q dikosongkan lalu diisi s.
10	Selama Q tidak kosong, jalankan langkah 11–18. Jika Q kosong, prosedur selesai.
11	Keluarkan simpul u dari depan antrean (FIFO).
12	Periksa setiap simpul v dalam $Adj[u]$, **sesuai urutan adjacency-list**. Jika semua sudah diperiksa, lanjut ke langkah 18.
13	Jika v masih WHITE (belum pernah ditemukan), jalankan langkah 14–17. Jika tidak, lanjut ke v berikutnya.
14–17	v diberi warna GRAY, jaraknya $u.d + 1$, predecessor-nya u, lalu v dimasukkan ke antrean.
18	Semua tetangga u sudah diperiksa: u diberi warna BLACK, lalu kembali ke langkah 10.

Tabel 2. Penjelasan langkah-langkah prosedur BFS.

CONTOH DAN PENERAPAN

3. Penelusuran BFS dari Simpul s

Contoh dari bahan kuliah: graf tak berarah dengan simpul r, s, t, u, v, w, x, y. Tentukan lintasan dari simpul s ke simpul-simpul lain dengan BFS.

Langkah pertama adalah menuliskan adjacency-list. Urutan berikut dari bahan kuliah, dan **urutan inilah yang menentukan** urutan simpul masuk antrean:

u	$Adj[u]$	u	$Adj[u]$
s	w → r	v	r
r	s → v	w	s → t → x
t	u → w → x	x	t → u → y → w
u	t → x → y	y	x → u

Tabel 3. Adjacency-list graf contoh.

3.1 Tabel penelusuran

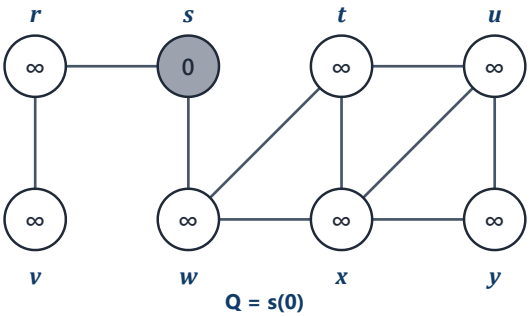
Putaran	u = DEQUEUE	Pemeriksaan $Adj[u]$	Ditemukan (warna GRAY)	Q setelah putaran
awal	-	-	$s.d = 0$	s
1	s	w (WHITE), r (WHITE)	$w.d = 1, w.\pi = s$; $r.d = 1, r.\pi = s$	w, r
2	w	s (BLACK), t (WHITE), x (WHITE)	$t.d = 2, t.\pi = w$; $x.d = 2, x.\pi = w$	r, t, x
3	r	s (BLACK), v (WHITE)	$v.d = 2, v.\pi = r$	t, x, v
4	t	u (WHITE), w (BLACK), x (GRAY)	$u.d = 3, u.\pi = t$	x, v, u
5	x	t (BLACK), u (GRAY), y (WHITE), w (BLACK)	$y.d = 3, y.\pi = x$	v, u, y
6	v	r (BLACK)	-	u, y
7	u	t (BLACK), x (BLACK), y (GRAY)	-	y
8	y	x (BLACK), u (BLACK)	-	kosong

Tabel 4. Setiap putaran while: simpul yang dikeluarkan, simpul WHITE yang ditemukan, dan isi antrean sesudahnya.

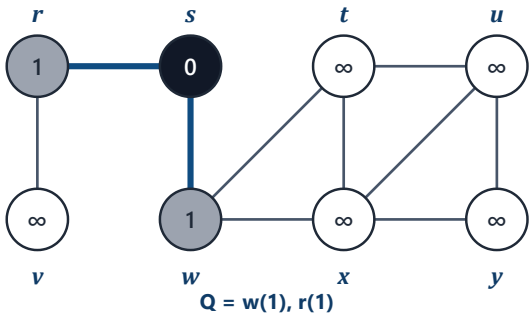
Setelah putaran 8, Q kosong sehingga kondisi langkah 10 bernilai salah dan prosedur BFS selesai. Perhatikan putaran 4: simpul x adalah tetangga t, tetapi sudah GRAY (ditemukan oleh w pada putaran 2), sehingga tidak diubah. Inilah yang mencegah sebuah simpul dimasukkan ke antrean lebih dari sekali.

3.2 Gambar keadaan graf

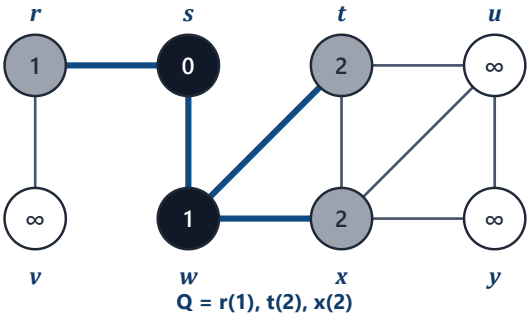
Gambar berikut memperlihatkan keadaan graf di awal dan setelah setiap putaran. Angka di dalam lingkaran adalah $u.d$; warna simpul menunjukkan $u.color$ (putih, abu-abu, hitam); busur tebal adalah busur predecessor ($v.\pi = u$); di bawah setiap gambar tertulis isi Q beserta jarak setiap simpul di dalamnya.



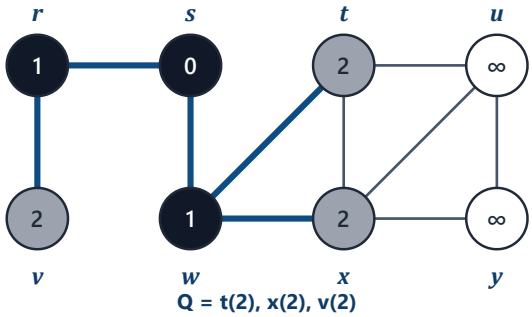
Gambar 1. (a) Setelah langkah 1–9 (inisialisasi).



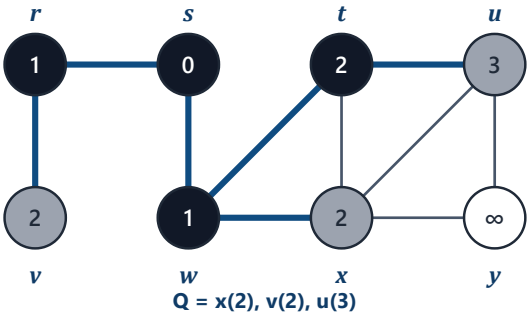
Gambar 2. (b) Setelah s dikeluarkan dari Q dan menjadi BLACK.



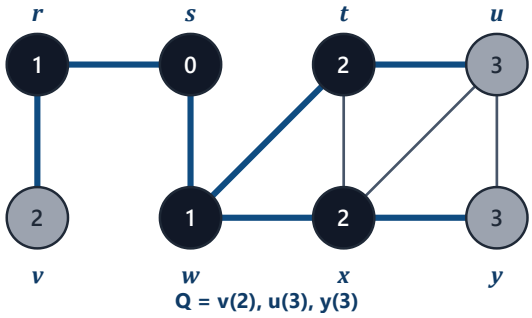
Gambar 3. (c) Setelah w dikeluarkan dari Q dan menjadi BLACK.



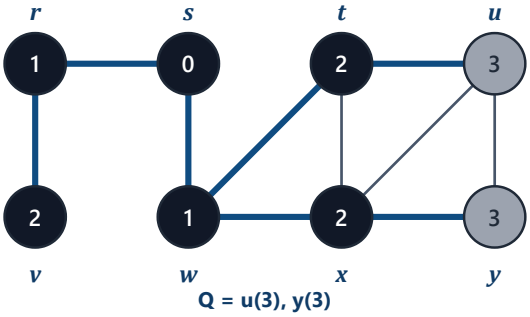
Gambar 4. (d) Setelah r dikeluarkan dari Q dan menjadi BLACK.



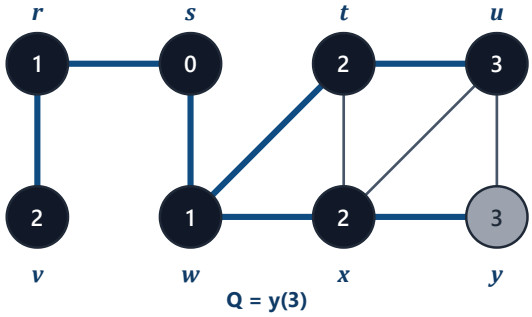
Gambar 5. (e) Setelah t dikeluarkan dari Q dan menjadi BLACK.



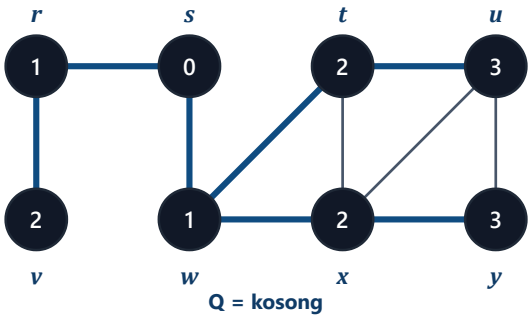
Gambar 6. (f) Setelah x dikeluarkan dari Q dan menjadi BLACK.



Gambar 7. (g) Setelah v dikeluarkan dari Q dan menjadi BLACK.



Gambar 8. (h) Setelah u dikeluarkan dari Q dan menjadi BLACK.



Gambar 9. (i) Setelah y dikeluarkan dari Q dan menjadi BLACK.

3.3 Hasil: lintasan dari s

Lintasan dari s ke setiap simpul v dibaca **mundur** mengikuti predecessor: $v, v.\pi, v.\pi.\pi, \ldots$ sampai s, lalu dibalik.

Tujuan	Lintasan	Jumlah busur ($v.d$)
r	s → r	1
w	s → w	1
t	s → w → t	2
x	s → w → x	2
v	s → r → v	2
u	s → w → t → u	3
y	s → w → x → y	3

Tabel 5. Lintasan dengan jumlah busur minimum dari s (sesuai bahan kuliah).

Busur-busur predecessor membentuk sebuah tree berakar di s yang disebut **BFS-tree** (pohon breadth-first). Tree ini memiliki 8 simpul dan 7 busur, sesuai sifat tree dari Modul 16.

PRINT-PATH(G, s, v) // mencetak lintasan s ke v (Cormen dkk., 2009)

```
1  if v == s
2      print s
3  elseif v.π == NIL
4      print "tidak ada lintasan dari" s "ke" v
5  else PRINT-PATH(G, s, v.π)
6      print v
```

3.4 Implementasi Python

Python: BFS dengan antrean deque

```python
from collections import deque
INF = float("inf")

def bfs(Adj, s):
    color = {u: "WHITE" for u in Adj}
    d = {u: INF for u in Adj}
    pi = {u: None for u in Adj}
    color[s], d[s] = "GRAY", 0
    Q = deque([s])
    while Q:
        u = Q.popleft()                    # DEQUEUE (FIFO)
        for v in Adj[u]:
            if color[v] == "WHITE":
                color[v], d[v], pi[v] = "GRAY", d[u] + 1, u
                Q.append(v)                # ENQUEUE
        color[u] = "BLACK"
    return d, pi

def lintasan(pi, s, v):
    return lintasan(pi, s, pi[v]) + [v] if v != s else [s]

Adj = {"s": ["w", "r"], "r": ["s", "v"], "t": ["u", "w", "x"],
       "u": ["t", "x", "y"], "v": ["r"], "w": ["s", "t", "x"],
       "x": ["t", "u", "y", "w"], "y": ["x", "u"]}
d, pi = bfs(Adj, "s")
for v in "rwtxvuy":
    print(v, d[v], " -> ".join(lintasan(pi, "s", v)))
```

Keluaran program

```
r 1 s -> r
w 1 s -> w
t 2 s -> w -> t
x 2 s -> w -> x
v 2 s -> r -> v
u 3 s -> w -> t -> u
y 3 s -> w -> x -> y
```

ANALISIS DAN HUBUNGAN KONSEP

4. Analisis Running Time dan Sifat BFS

4.1 Running time

- **Inisialisasi (langkah 1–9)**: setiap simpul diproses sekali, sehingga $O(V)$.
- **Operasi antrean**: setiap simpul di-ENQUEUE paling banyak sekali (hanya saat masih WHITE) dan di-DEQUEUE paling banyak sekali. Setiap operasi $O(1)$, total $O(V)$.
- **Pemeriksaan adjacency-list (langkah 12–17)**: list setiap simpul ditelusuri satu kali, yaitu ketika simpul itu di-DEQUEUE. Total panjang semua list adalah $\Theta(E)$, sehingga totalnya $O(E)$.

Jadi running time BFS adalah $O(V + E)$, linier terhadap ukuran representasi adjacency-list. Jika graf disimpan sebagai adjacency-matrix, pemeriksaan tetangga setiap simpul membutuhkan $\Theta(V)$, sehingga total menjadi $O(V^2)$ (Modul 16).

4.2 Sifat penting

- **Jarak terpendek dalam jumlah busur.** Untuk setiap simpul v yang dapat dicapai dari s, nilai $v.d$ di akhir BFS sama dengan banyaknya busur paling sedikit pada lintasan dari s ke v (Cormen dkk., 2009, Teorema 22.5). Simpul yang tidak dapat dicapai tetap bernilai ∞.
- **Tidak memperhitungkan bobot.** BFS menganggap setiap busur bernilai 1. Untuk graf berbobot, lintasan dengan busur paling sedikit belum tentu berbobot paling kecil; untuk itu dibutuhkan Bellman-Ford atau Dijkstra (Modul 22–24). Dijkstra dapat dipandang sebagai perumuman BFS yang memakai antrean prioritas, bukan antrean FIFO.
- **BFS-tree bergantung pada urutan adjacency-list.** Jarak $v.d$ selalu sama, tetapi jika urutan list diubah, predecessor dapat berbeda. Misalnya jika $Adj[x]$ diproses sebelum $Adj[t]$, simpul u bisa saja ditemukan oleh x sehingga lintasannya $s \to w \to x \to u$, tetap 3 busur.
- **Hanya simpul yang terjangkau.** BFS dari s hanya menemukan simpul yang terhubung dengan s; hasilnya satu tree. DFS (Modul 18) sebaliknya menelusuri seluruh simpul dan dapat menghasilkan beberapa tree.

BFS sering dikaitkan dengan Moore (1959), yang memakai penelusuran melebar untuk mencari lintasan terpendek di dalam labirin.

KESIMPULAN

5. Kesimpulan

- BFS menelusuri graf dari simpul awal s lapis demi lapis menggunakan antrean FIFO.
- Warna WHITE/GRAY/BLACK menandai simpul belum ditemukan / di antrean / selesai; $u.d$ menyimpan jarak (jumlah busur) dan $u.\pi$ menyimpan predecessor.
- Setiap simpul masuk antrean paling banyak sekali; running time BFS $O(V + E)$ dengan adjacency-list.
- $v.d$ adalah jumlah busur minimum dari s ke v; lintasannya disusun mundur lewat predecessor dan membentuk BFS-tree.
- Pada contoh bahan kuliah: r, w berjarak 1; t, x, v berjarak 2; u, y berjarak 3.

Kuis Pemahaman

Kerjakan semua soal terlebih dahulu tanpa melihat kunci jawaban. Kunci jawaban beserta pembahasannya ada di halaman berikutnya.

1 PILIHAN GANDA

Struktur data apa yang membuat BFS menelusuri graf secara melebar?

A. Tumpukan (stack, LIFO)
B. Antrean (queue, FIFO)
C. Antrean prioritas
D. Array terurut

2 PENELUSURAN

Jalankan BFS dari simpul 1 pada graf tak berarah Modul 16 dengan adjacency-list $1 : 2, 5$; $2 : 1, 5, 3, 4$; $3 : 2, 4$; $4 : 2, 5, 3$; $5 : 4, 1, 2$. Tuliskan urutan DEQUEUE serta d dan π setiap simpul.

3 ANALISIS

Pada penelusuran di modul ini, mengapa simpul u tidak dimasukkan ke antrean untuk kedua kalinya ketika x diproses pada putaran 5?

4 BENAR/SALAH DENGAN ALASAN

"Jika urutan adjacency-list diubah, nilai $v.d$ hasil BFS dapat berubah." Benar atau salah?

5 PERHITUNGAN

Sebuah graf berarah memiliki 1.000 simpul dan 5.000 busur, disimpan sebagai adjacency-list. Berapa kira-kira banyaknya operasi ENQUEUE paling banyak, dan berapa banyak pemeriksaan elemen adjacency-list paling banyak yang dilakukan BFS?

6 ANALISIS KASUS

Pada graf berbobot, busur $s \to a$ berbobot 10, sedangkan $s \to b$ dan $b \to a$ masing-masing berbobot 1. Lintasan apa yang ditemukan BFS dari s ke a? Apakah lintasan itu berbobot minimum?

7 PILIHAN GANDA

Simpul yang berwarna GRAY pada suatu saat dalam BFS adalah simpul yang ...

A. Belum ditemukan.
B. Sudah ditemukan dan masih berada di antrean.
C. Sudah diperiksa semua tetangganya.
D. Tidak dapat dicapai dari s.

8 PENYUSUNAN LINTASAN

Dengan hasil BFS pada modul ini, gunakan predecessor untuk menyusun lintasan dari s ke y langkah demi langkah.

Kunci Jawaban dan Pembahasan

Bandingkan jawabanmu, lalu baca pembahasannya untuk memahami alasan di balik setiap jawaban.

Soal 1

Jawaban: B. FIFO menjamin simpul berjarak k dikeluarkan sebelum simpul berjarak $k+1$. Stack (A) menghasilkan penelusuran mendalam seperti DFS. Antrean prioritas (C) dipakai Dijkstra dan Prim.

Soal 2

DEQUEUE 1: temukan 2 dan 5 ($d = 1$, $\pi = 1$); $Q = 2, 5$. DEQUEUE 2: 1 dan 5 sudah tidak WHITE; temukan 3 dan 4 ($d = 2$, $\pi = 2$); $Q = 5, 3, 4$. DEQUEUE 5, 3, 4: tidak ada simpul WHITE baru. Urutan DEQUEUE: **1, 2, 5, 3, 4**. Hasil: $d(1) = 0$; $d(2) = d(5) = 1$ dengan $\pi = 1$; $d(3) = d(4) = 2$ dengan $\pi = 2$.

Soal 3

Saat x diproses, u sudah berwarna **GRAY** karena ditemukan oleh t pada putaran 4. Langkah 13 hanya memproses simpul WHITE, sehingga u tidak diubah dan tidak dimasukkan lagi ke antrean.

Soal 4

Salah. $v.d$ selalu sama dengan jumlah busur minimum dari s ke v, berapa pun urutan list. Yang dapat berubah adalah predecessor $v.\pi$ (bentuk BFS-tree) bila ada beberapa lintasan terpendek.

Soal 5

ENQUEUE paling banyak **1.000** kali (sekali per simpul, termasuk s). Pemeriksaan elemen list paling banyak **5.000** (setiap busur berarah muncul sekali di list). Total kerja sebanding dengan $V + E = 6.000$.

Soal 6

BFS memilih $s \to a$ karena hanya 1 busur ($a.d = 1$). Bobotnya 10, padahal $s \to b \to a$ berbobot 2. Jadi lintasan BFS **tidak** berbobot minimum; BFS mengabaikan bobot. Untuk graf berbobot dibutuhkan Dijkstra atau Bellman-Ford.

Soal 7

Jawaban: B. WHITE = belum ditemukan (A), BLACK = selesai (C). Simpul yang tidak terjangkau tetap WHITE (D).

Soal 8

$y.\pi = x$, $x.\pi = w$, $w.\pi = s$. Barisan mundur: y, x, w, s. Dibalik: $s \to w \to x \to y$, dengan 3 busur ($y.d = 3$).

Referensi

Sumber utama: Materi 11.1 ALGORITMA GRAF, BFS (bahan kuliah Perancangan dan Analisis Algoritma). Penjelasan tambahan, koreksi, dan pengayaan mengacu pada pustaka berikut.

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). Cambridge, MA: MIT Press. ISBN 978-0-262-03384-8.

2. Moore, E. F. (1959). The Shortest Path Through a Maze. Dalam *Proceedings of the International Symposium on the Theory of Switching* (hlm. 285–292). Cambridge, MA: Harvard University Press.
3. Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Upper Saddle River, NJ: Addison-Wesley.